# CSA3710 SOFTWARE TESTING
# EXPERIMENTS

**Name:** Kaviya V **Register No:** 192421022

**QUESTIONS:**

**10. Develop a comprehensive test plan document for a chosen application, such as a Library Management System, outlining the various testing strategies, methodologies, and criteria to ensure thorough evaluation and validation of the system's functionality.**

**Code:**

```
package npkb;
import java.util.Scanner;

public class npkb {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      String username, password, bookName;
      int choice;
      int totalBooks = 0;
      int borrowedBooks = 0;

      System.out.println("===== LIBRARY MANAGEMENT SYSTEM =====");

      do {
         System.out.println("\n1. User Registration / Login");
         System.out.println("2. Search for a Book");
         System.out.println("3. Borrow a Book");
         System.out.println("4. Return a Book");
         System.out.println("5. Add a New Book");
         System.out.println("6. Display Number of Books");
         System.out.println("7. Generate Borrowing Report");
         System.out.println("8. Display Book ID");
         System.out.println("9. Security Settings");
         System.out.println("10. Membership Renewal / Payment");
```

```
System.out.println("0. Exit");

System.out.print("\nEnter your choice: ");
choice = sc.nextInt();
sc.nextLine();

switch (choice) {

    case 1:
        System.out.print("Enter username: ");
        username = sc.nextLine();

        System.out.print("Enter password: ");
        password = sc.nextLine();

        if (!username.isEmpty() && !password.isEmpty()) {
            System.out.println("Login successful!");
        } else {
            System.out.println("Invalid username or password!");
        }
        break;

    case 2:
        System.out.print("Enter book name to search: ");
        bookName = sc.nextLine();
        System.out.println("Book search completed for: " + bookName);
        break;

    case 3:
        if (totalBooks > borrowedBooks) {
            System.out.print("Enter book name to borrow: ");
            bookName = sc.nextLine();
            borrowedBooks++;
            System.out.println("Book borrowed successfully!");
        } else {
            System.out.println("No books available to borrow.");
        }
        break;

    case 4:
```

```
    if (borrowedBooks > 0) {
        System.out.print("Enter book name to return: ");
        bookName = sc.nextLine();
        borrowedBooks--;
        System.out.println("Book returned successfully!");
    } else {
        System.out.println("No borrowed books to return.");
    }
    break;

case 5:
    System.out.print("Enter number of books to add: ");
    int count = sc.nextInt();

    if (count > 0) {
        totalBooks += count;
        System.out.println("Books added successfully!");
    } else {
        System.out.println("Invalid book count!");
    }
    break;

case 6:
    System.out.println("Total books: " + totalBooks);
    System.out.println("Borrowed books: " + borrowedBooks);
    System.out.println("Available books: "
            + (totalBooks - borrowedBooks));
    break;

case 7:
    System.out.println("\n===== BORROWING REPORT =====");
    System.out.println("Total books: " + totalBooks);
    System.out.println("Borrowed books: " + borrowedBooks);
    System.out.println("Available books: "
            + (totalBooks - borrowedBooks));
    break;

case 8:
    System.out.print("Enter book name: ");
    bookName = sc.nextLine();
```

```
                System.out.println("Book ID: LIB101");
                System.out.println("Book name: " + bookName);
                break;

            case 9:
                System.out.println("===== SECURITY SETTINGS =====");
                System.out.println("Password protection enabled.");
                System.out.println("User authentication required.");
                break;

            case 10:
                System.out.println("1. Membership Renewal");
                System.out.println("2. Payment");
                System.out.print("Enter option: ");
                int paymentChoice = sc.nextInt();

                if (paymentChoice == 1) {
                    System.out.println("Membership renewed successfully!");
                } else if (paymentChoice == 2) {
                    System.out.println("Payment completed successfully!");
                } else {
                    System.out.println("Invalid payment option!");
                }
                break;

            case 0:
                System.out.println("Exiting Library Management System.");
                break;

            default:
                System.out.println("Invalid choice! Try again.");
        }

    } while (choice != 0);

    sc.close();
  }
}
```

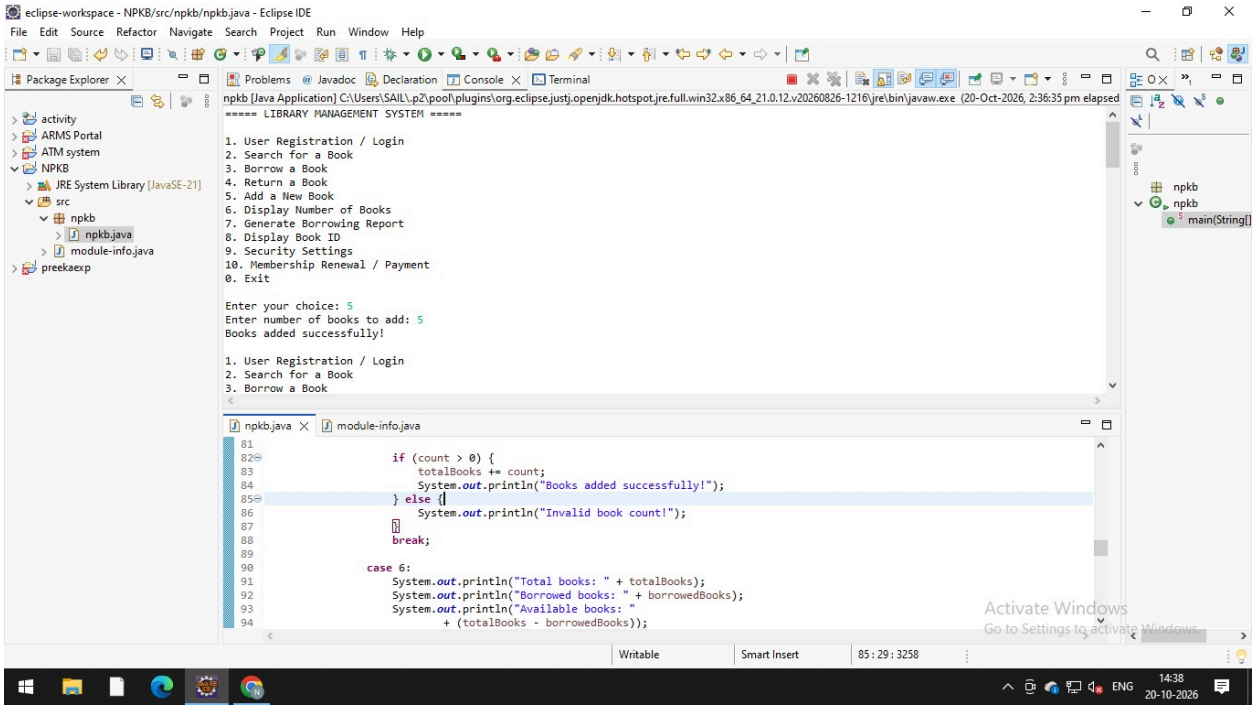
**Report table:**

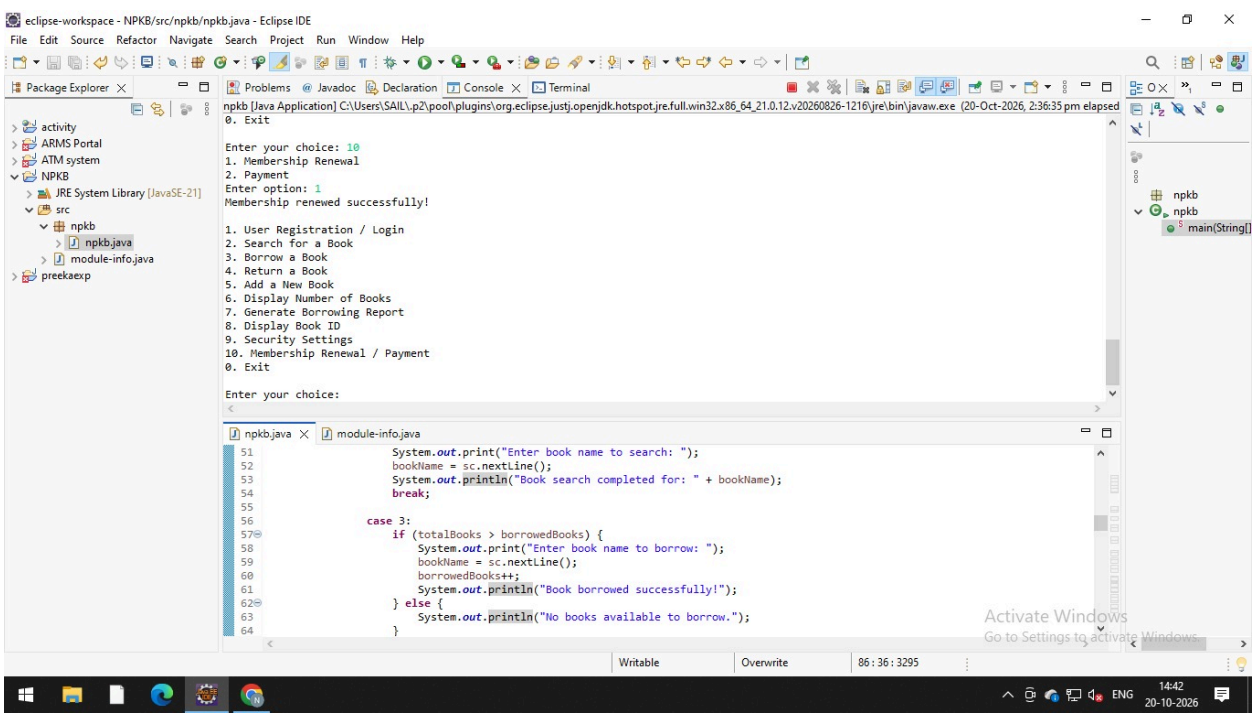
| S. No. | Feature | Test Scenario | Expected Result |
|---|---|---|---|
| 1 | Login | Enter valid username and password | User logs in successfully |
| 2 | Book Search | Search for an existing book | Correct book details are displayed |
| 3 | Borrow Book | Borrow an available book | Book is issued successfully |
| 4 | Return Book | Return an issued book | Book status is updated |
| 5 | Fine Calculation | Return a book after the due date | Fine is calculated correctly |
| 6 | Availability | Check the availability of a book | Correct availability is displayed |

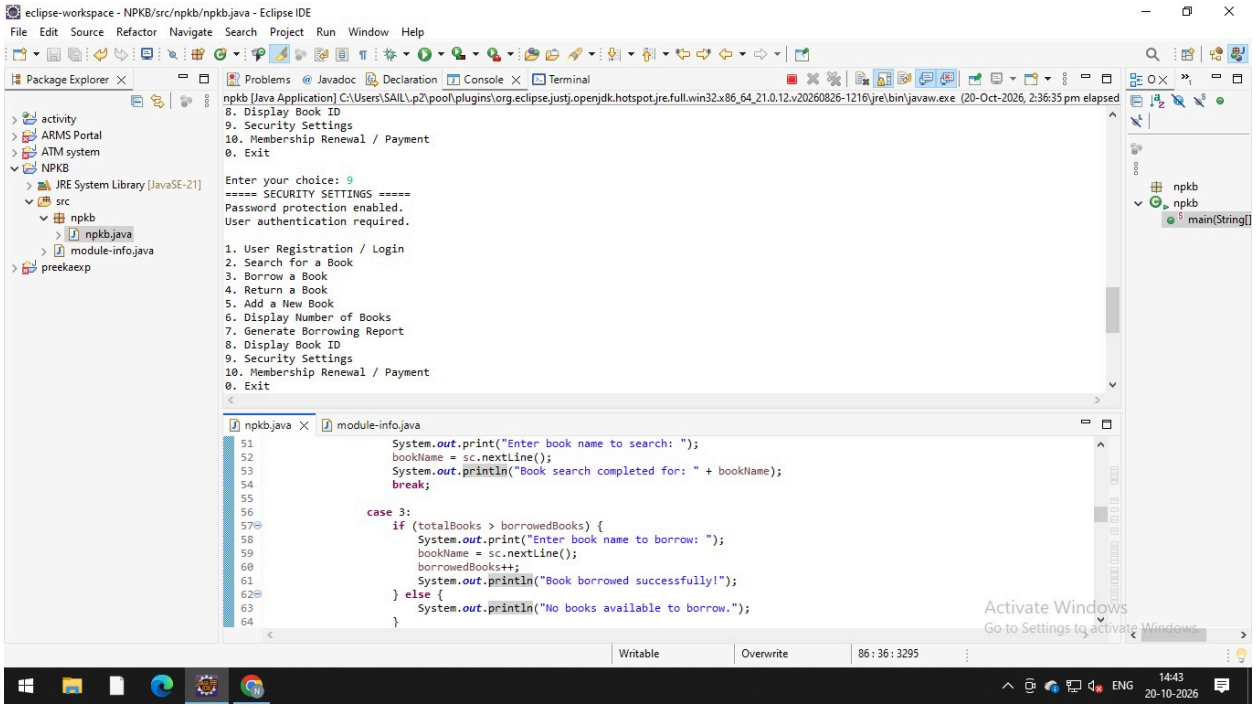
**Testing Approach:** Functional testing, validation testing, usability testing, integration testing, and regression testing.

**Result:** A comprehensive test plan was prepared for the major functions of the Library Management System.

# Code output:


eclipse-workspace - NPKB/src/npkb/npkb.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
Problems Javadoc Declaration Console Terminal
npkb [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20260826-1216\jre\bin\javaw.exe (20-Oct-2026, 2:36:35 pm elapsed
===== LIBRARY MANAGEMENT SYSTEM =====
1. User Registration / Login
2. Search for a Book
3. Borrow a Book
4. Return a Book
5. Add a New Book
6. Display Number of Books
7. Generate Borrowing Report
8. Display Book ID
9. Security Settings
10. Membership Renewal / Payment
0. Exit
Enter your choice: 5
Enter number of books to add: 5
Books added successfully!
1. User Registration / Login
2. Search for a Book
3. Borrow a Book
npkb.java
module-info.java
if (count > 0) {
totalBooks += count;
System.out.println("Books added successfully!");
} else {
System.out.println("Invalid book count!");
}
break;
case 6:
System.out.println("Total books: " + totalBooks);
System.out.println("Borrowed books: " + borrowedBooks);
System.out.println("Available books: "
+ (totalBooks - borrowedBooks));
Activate Windows
Go to Settings to activate Windows.
Writable
Smart Insert
85 : 29 : 3258
ENG
14:38
20-10-2026


eclipse-workspace - NPKB/src/npkb/npkb.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
Problems Javadoc Declaration Console Terminal
npkb [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20260826-1216\jre\bin\javaw.exe (20-Oct-2026, 2:36:35 pm elapsed
0. Exit
Enter your choice: 10
1. Membership Renewal
2. Payment
Enter option: 1
Membership renewed successfully!
1. User Registration / Login
2. Search for a Book
3. Borrow a Book
4. Return a Book
5. Add a New Book
6. Display Number of Books
7. Generate Borrowing Report
8. Display Book ID
9. Security Settings
10. Membership Renewal / Payment
0. Exit
Enter your choice:
npkb.java
module-info.java
System.out.print("Enter book name to search: ");
bookName = sc.nextLine();
System.out.println("Book search completed for: " + bookName);
break;
case 3:
if (totalBooks > borrowedBooks) {
System.out.print("Enter book name to borrow: ");
bookName = sc.nextLine();
borrowedBooks++;
System.out.println("Book borrowed successfully!");
} else {
System.out.println("No books available to borrow.");
}
Activate Windows
Go to Settings to activate Windows.
Writable
Overwrite
86 : 36 : 3295
ENG
14:42
20-10-2026


eclipse-workspace - NPKB/src/npkb/npkb.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
activity
ARMS Portal
ATM system
NPKB
JRE System Library [JavaSE-21]
src
npkb
npkb.java
module-info.java
preekaexp
Problems Javadoc Declaration Console Terminal
npkb [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20260826-1216\jre\bin\javaw.exe (20-Oct-2026, 2:36:35 pm elapsed
8. Display Book ID
9. Security Settings
10. Membership Renewal / Payment
0. Exit
Enter your choice: 9
===== SECURITY SETTINGS =====
Password protection enabled.
User authentication required.
1. User Registration / Login
2. Search for a Book
3. Borrow a Book
4. Return a Book
5. Add a New Book
6. Display Number of Books
7. Generate Borrowing Report
8. Display Book ID
9. Security Settings
10. Membership Renewal / Payment
0. Exit
npkb.java
module-info.java
System.out.print("Enter book name to search: ");
bookName = sc.nextLine();
System.out.println("Book search completed for: " + bookName);
break;
case 3:
if (totalBooks > borrowedBooks) {
System.out.print("Enter book name to borrow: ");
bookName = sc.nextLine();
borrowedBooks++;
System.out.println("Book borrowed successfully!");
} else {
System.out.println("No books available to borrow.");
}
Activate Windows
Go to Settings to activate Windows.
Writable
Overwrite
86 : 36 : 3295
ENG
14:43
20-10-2026

**11. Write a case study to identify and analyze the requirements of a Railway Reservation System and prepare positive test scenarios and test cases for the application.**

**Code:**

```
package npkb;
import java.util.Scanner;
import java.time.LocalDate;
import java.util.Random;

public class npkb{

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        Random random = new Random();

        String username = "";
        String password = "";
        String source = "";
        String destination = "";
        String journeyDate = "";
        String travelClass = "";
        String passengerName = "";
        String berth = "";
        boolean registered = false;
        boolean loggedIn = false;
        boolean paymentDone = false;
        String pnr = "";

        int choice;

        System.out.println("===== RAILWAY RESERVATION SYSTEM =====");

        do {
            System.out.println("\n1. User Registration");
            System.out.println("2. User Login");
            System.out.println("3. Train Search");
            System.out.println("4. Journey Date Selection");
            System.out.println("5. Class Selection");
            System.out.println("6. Passenger Details");
```

```
System.out.println("7. Berth Preference");
System.out.println("8. Payment");
System.out.println("9. PNR Generation");
System.out.println("10. Ticket Confirmation");
System.out.println("0. Exit");
System.out.print("\nEnter your choice: ");
choice = sc.nextInt();
sc.nextLine();
switch (choice) {

    case 1:
        System.out.print("Enter username: ");
        username = sc.nextLine();

        System.out.print("Enter email: ");
        String email = sc.nextLine();

        System.out.print("Enter 10-digit mobile number: ");
        String mobile = sc.nextLine();

        System.out.print("Create password: ");
        password = sc.nextLine();

        if (!username.isEmpty()
                && email.contains("@")
                && mobile.matches("[0-9]{10}")
                && !password.isEmpty()) {
            registered = true;
            System.out.println("Registration successful!");
        } else {
            System.out.println("Enter valid registration details.");
        }
        break;

    case 2:
        System.out.print("Enter username: ");
        String loginUser = sc.nextLine();

        System.out.print("Enter password: ");
        String loginPass = sc.nextLine();
```

```
    if (registered && loginUser.equals(username)
            && loginPass.equals(password)) {
        loggedIn = true;
        System.out.println("Login successful!");
    } else {
        System.out.println("Please register or enter correct credentials.");
    }
    break;

case 3:
    if (!loggedIn) {
        System.out.println("Please login first.");
        break;
    }
    try {
        LocalDate date = LocalDate.parse(journeyDate);

        if (date.isAfter(LocalDate.now())) {
            System.out.println("Journey date selected successfully.");
        } else {
            System.out.println("Select a future journey date.");
        }
    } catch (Exception e) {
        System.out.println("Invalid date format.");
    }
    break;

case 5:
    System.out.println("Select class:");
    System.out.println("1. SL");
    System.out.println("2. 3A");
    System.out.println("3. 2A");
    System.out.println("4. 1A");
    System.out.print("Enter choice: ");

    int classChoice = sc.nextInt();
    sc.nextLine();

    switch (classChoice) {
```

```
            case 1: travelClass = "SL"; break;
            case 2: travelClass = "3A"; break;
            case 3: travelClass = "2A"; break;
            case 4: travelClass = "1A"; break;
            default:
                travelClass = "";
                System.out.println("Invalid class selection.");
        }

        if (!travelClass.isEmpty()) {
            System.out.println("Selected class: " + travelClass);
            System.out.println("Availability: Available (Demo)");
        }
        break;
      case 7:
        System.out.println("1. Lower");
        System.out.println("2. Middle");
        System.out.println("3. Upper");
        System.out.print("Select berth preference: ");

        int berthChoice = sc.nextInt();
        sc.nextLine();

        switch (berthChoice) {
            case 1: berth = "Lower"; break;
            case 2: berth = "Middle"; break;
            case 3: berth = "Upper"; break;
            default: berth = "";
        }

        if (!berth.isEmpty()) {
            System.out.println("Berth preference saved: " + berth);
        } else {
            System.out.println("Invalid berth preference.");
        }
        break;

    case 8:
        System.out.println("Select payment method:");
        System.out.println("1. UPI");
```

```
            System.out.println("2. Net Banking");
            System.out.print("Enter choice: ");

            int payChoice = sc.nextInt();
            sc.nextLine();

            if (payChoice == 1 || payChoice == 2) {
                paymentDone = true;
                System.out.println("Demo payment successful!");
            } else {
                System.out.println("Invalid payment method.");
            }
            break;

        case 9:
            if (paymentDone) {
                pnr = String.format("%010d",
                        random.nextLong(1, 10_000_000_000L));
                System.out.println("PNR generated successfully: " + pnr);
            } else {
                System.out.println("Complete payment before PNR generation.");
            }
            break;

        case 10:
            if (paymentDone && !pnr.isEmpty()) {
                System.out.println("===== TICKET CONFIRMATION =====");
                System.out.println("Passenger: " + passengerName);
                System.out.println("Source: " + source);
                System.out.println("Destination: " + destination);
                System.out.println("Journey Date: " + journeyDate);
                System.out.println("Class: " + travelClass);
                System.out.println("Berth: " + berth);
                System.out.println("PNR: " + pnr);
                System.out.println("Ticket confirmed (Demo).");
                System.out.println("PDF/SMS service is simulated.");
            } else {
                System.out.println("Complete payment and generate PNR first.");
            }
            break;
```

```
            case 0:
                System.out.println("Thank you for using Railway Reservation System.");
                break;

            default:
                System.out.println("Invalid choice. Try again.");
        }

    } while (choice != 0);

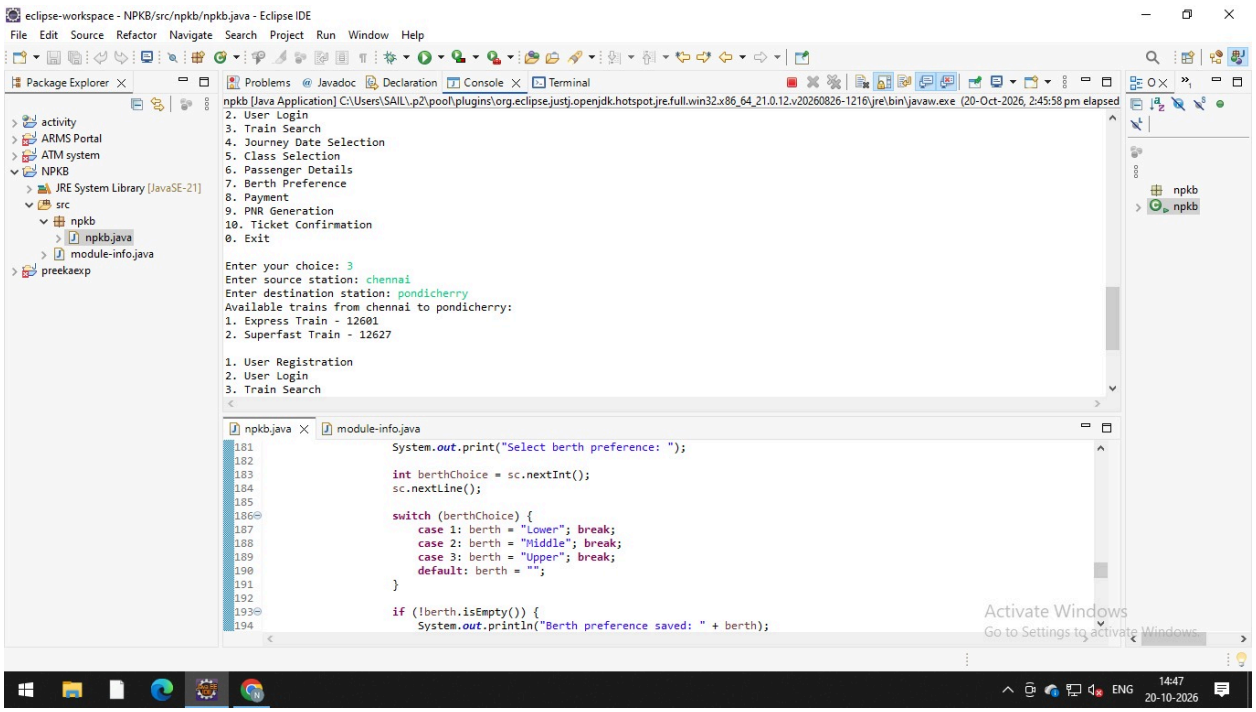
    sc.close();
  }
}
```

**Report:**

| S. No. | Feature | Test Scenario | Expected Result |
|---|---|---|---|
| 1 | Login | Enter valid login credentials | User logs in successfully |
| 2 | Train Search | Enter valid source and destination | Available trains are displayed |
| 3 | Journey Date | Select a valid future journey date | Trains for the selected date are displayed |
| 4 | Seat Availability | Select a train and class | Seat availability is displayed |
| 5 | Passenger Details | Enter valid passenger information | Details are accepted |
| 6 | Ticket Booking | Confirm booking with valid details | Ticket is booked and PNR is generated |
| 7 | Payment | Complete payment successfully | Booking is confirmed |
| 8 | Booking History | View an existing booking | Ticket and PNR details are displayed |

Result: Positive test scenarios and test cases were prepared for the main functions of the Railway Reservation System.

## Code output:



**12. To find the Nth term of the Fibonacci series and validate the output using JUnit assertions.**

**Code:**

```
package npkb;
import java.util.Scanner;
import java.time.LocalDate;

public class npkb{

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);
      int choice;

      do {
         System.out.println("\n===== RAILWAY RESERVATION SYSTEM =====");
         System.out.println("1. User Registration - Invalid Email");
         System.out.println("2. User Login - Wrong Password");
         System.out.println("3. Train Search - Same Station");
         System.out.println("4. Journey Date - Past Date");
         System.out.println("5. Passenger Age Validation");
         System.out.println("6. Passenger Name Validation");
         System.out.println("7. Passenger Booking Limit");
         System.out.println("8. Payment Failure");
         System.out.println("9. PNR Length Validation");
         System.out.println("10. Ticket Cancellation After Departure");
         System.out.println("0. Exit");

         System.out.print("Enter your choice: ");
         choice = sc.nextInt();
         sc.nextLine();

         switch (choice) {

            case 1:
               System.out.print("Enter email: ");
               String email = sc.nextLine();

               if (!email.matches("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$")) {
```

```
            System.out.println("Error: Invalid email format.");
        } else {
            System.out.println("Email format is valid.");
        }
        break;

    case 2:
        String correctPassword = "train123";

        System.out.print("Enter password: ");
        String password = sc.nextLine();

        if (!password.equals(correctPassword)) {
            System.out.println("Invalid Credentials");
            System.out.println("Login denied.");
        } else {
            System.out.println("Login successful.");
        }
        break;

    case 3:
        System.out.print("Enter source station: ");
        String source = sc.nextLine();

        System.out.print("Enter destination station: ");
        String destination = sc.nextLine();

        if (source.equalsIgnoreCase(destination)) {
            System.out.println(
                "Error: Source and destination cannot be the same.");
        } else {
            System.out.println("Route details accepted.");
        }
        break;

    case 4:
        System.out.print("Enter journey date (YYYY-MM-DD): ");
        String dateInput = sc.nextLine();

        try {
```

```
            LocalDate journeyDate = LocalDate.parse(dateInput);

            if (journeyDate.isBefore(LocalDate.now())) {
                System.out.println(
                    "Error: Past dates are not allowed.");
            } else {
                System.out.println("Journey date accepted.");
            }
        } catch (Exception e) {
            System.out.println("Error: Invalid date format.");
        }
        break;

    case 5:
        System.out.print("Enter passenger age: ");

        try {
            int age = Integer.parseInt(sc.nextLine());

            if (age <= 0) {
                System.out.println(
                    "Error: Age must be greater than zero.");
            } else {
                System.out.println("Passenger age accepted.");
            }
        } catch (NumberFormatException e) {
            System.out.println("Error: Enter a valid numeric age.");
        }
        break;

    case 6:
        System.out.print("Enter passenger name: ");
        String name = sc.nextLine();

        if (!name.matches("[A-Za-z ]+")) {
            System.out.println(
                "Error: Name must contain only letters and spaces.");
        } else {
            System.out.println("Passenger name accepted.");
        }
```

```
            break;

        case 7:
            System.out.print("Enter number of passengers: ");

            try {
                int passengers = Integer.parseInt(sc.nextLine());

                if (passengers > 6) {
                    System.out.println(
                        "Error: Maximum 6 passengers allowed.");
                } else if (passengers <= 0) {
                    System.out.println(
                        "Error: At least one passenger is required.");
                } else {
                    System.out.println(
                        "Passenger count accepted: " + passengers);
                }
            } catch (NumberFormatException e) {
                System.out.println("Error: Enter a valid number.");
            }
            break;

        case 8:
            System.out.println("Enter payment card details (demo):");
            System.out.print("Is the card expired? (yes/no): ");
            String expired = sc.nextLine();

            if (expired.equalsIgnoreCase("yes")) {
                System.out.println("Transaction Failed");
                System.out.println("Please use a valid payment method.");
            } else if (expired.equalsIgnoreCase("no")) {
                System.out.println("Demo payment successful.");
            } else {
                System.out.println("Invalid input.");
            }
            break;

        case 9:
            System.out.print("Enter PNR number: ");
```

```
                String pnr = sc.nextLine();

                if (!pnr.matches("[0-9]{10}")) {
                    System.out.println("Invalid PNR");
                } else {
                    System.out.println("PNR format is valid.");
                }
                break;

            case 10:
                System.out.print(
                    "Has the train already departed? (yes/no): ");
                String departed = sc.nextLine();

                if (departed.equalsIgnoreCase("yes")) {
                    System.out.println(
                        "Error: Ticket cancellation is not allowed after departure.");
                } else if (departed.equalsIgnoreCase("no")) {
                    System.out.println(
                        "Ticket cancellation can proceed (demo).");
                } else {
                    System.out.println("Invalid input.");
                }
                break;

            case 0:
                System.out.println("Exiting Railway Negative Test System.");
                break;

            default:
                System.out.println("Invalid choice. Try again.");
        }

    } while (choice != 0);

    sc.close();
  }
}
```

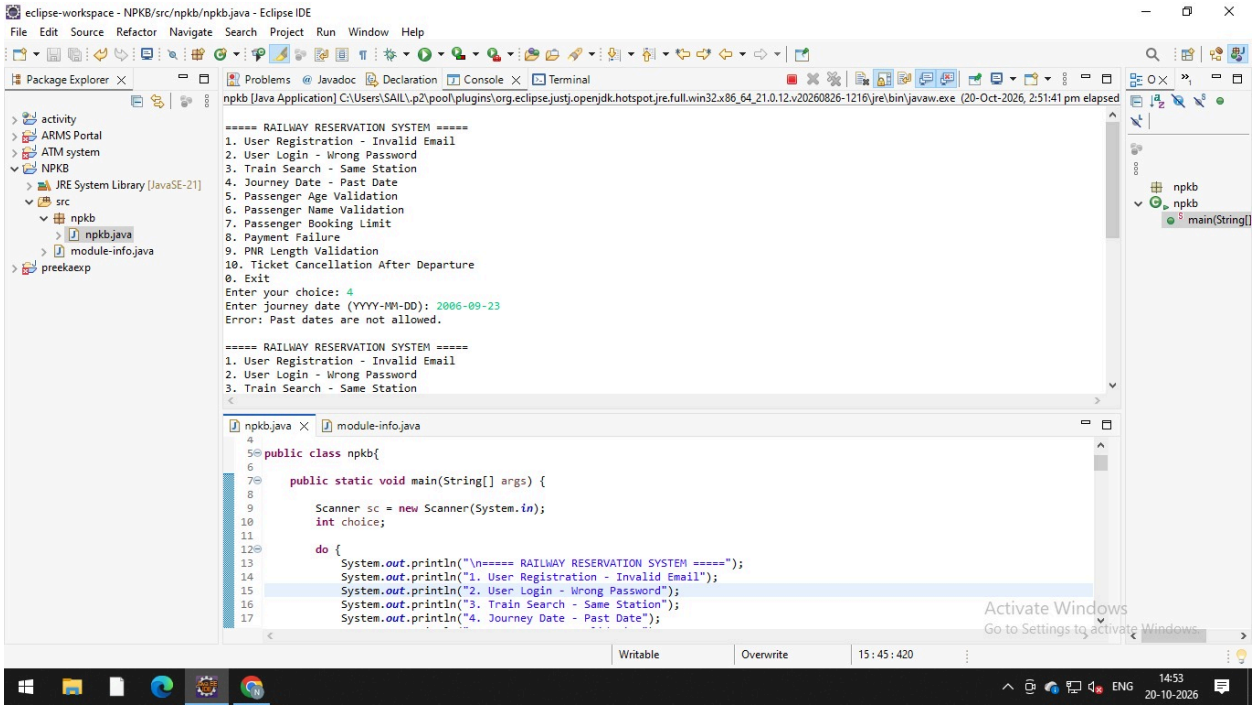
**Report:**

| S. No. | Test Scenario | Input(N) | Expected Result | JUnit Assertion |
|---|---|---|---|---|
| 1 | First term | 1 | 0 | assertEquals(0, fibonacci(1)); |
| 2 | Second term | 2 | 1 | assertEquals(1, fibonacci(2)); |
| 3 | Third term | 3 | 1 | assertEquals(1, fibonacci(3)); |
| 4 | Fifth term | 5 | 3 | assertEquals(3, fibonacci(5)); |
| 5 | Seventh term | 7 | 8 | assertEquals(8, fibonacci(7)); |
| 6 | Tenth term | 10 | 34 | assertEquals(34, fibonacci(10)); |
| 7 | Invalid input | 0 | Validation error or exception | assertThrows(...) |

Note: The assertThrows(...) syntax depends on the JUnit version. The expected results above are test expectations; mark tests as passed only after running them.

Result: Fibonacci test cases were prepared to validate the output for different inputs using JUnit assertions.

# Code output:


eclipse-workspace - NPKB/src/npkb/npkb.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
activity
ARMS Portal
ATM system
NPKB
JRE System Library [JavaSE-21]
src
npkb
npkb.java
module-info.java
preekaexp
Problems Javadoc Declaration Console Terminal
npkb [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.12.v20260826-1216\jre\bin\javaw.exe (20-Oct-2026, 2:51:41 pm elapsed
===== RAILWAY RESERVATION SYSTEM =====
1. User Registration - Invalid Email
2. User Login - Wrong Password
3. Train Search - Same Station
4. Journey Date - Past Date
5. Passenger Age Validation
6. Passenger Name Validation
7. Passenger Booking Limit
8. Payment Failure
9. PNR Length Validation
10. Ticket Cancellation After Departure
0. Exit
Enter your choice: 4
Enter journey date (YYYY-MM-DD): 2006-09-23
Error: Past dates are not allowed.
===== RAILWAY RESERVATION SYSTEM =====
1. User Registration - Invalid Email
2. User Login - Wrong Password
3. Train Search - Same Station
npkb.java
module-info.java
public class npkb{
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int choice;
do {
System.out.println("\n===== RAILWAY RESERVATION SYSTEM =====");
System.out.println("1. User Registration - Invalid Email");
System.out.println("2. User Login - Wrong Password");
System.out.println("3. Train Search - Same Station");
System.out.println("4. Journey Date - Past Date");
Activate Windows
Go to Settings to activate Windows.
Writable
Overwrite
15 : 45 : 420
ENG
14:53
20-10-2026